JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ReentrantLock

java.lang.Object
    java.util.concurrent.locks.ReentrantLock

All Implemented Interfaces:

`Serializable`, `Lock`

```
public class ReentrantLock
extends Object
implements Lock, Serializable
```

A reentrant mutual exclusion `Lock` with the same basic behavior and semantics as the implicit monitor lock accessed using `synchronized` methods and statements, but with extended capabilities.

A `ReentrantLock` is *owned* by the thread last successfully locking, but not yet unlocking it. A thread invoking `lock` will return, successfully acquiring the lock, when the lock is not owned by another thread. The method will return immediately if the current thread already owns the lock. This can be checked using methods `isHeldByCurrentThread()`, and `getHoldCount()`.

The constructor for this class accepts an optional *fairness* parameter. When set `true`, under contention, locks favor granting access to the longest-waiting thread. Otherwise this lock does not guarantee any particular access order. Programs using fair locks accessed by many threads may display lower overall throughput (i.e., are slower; often much slower) than those using the default setting, but have smaller variances in times to obtain locks and guarantee lack of starvation. Note however, that fairness of locks does not guarantee fairness of thread scheduling. Thus, one of many threads using a fair lock may obtain it multiple times in succession while other active threads are not progressing and not currently holding the lock. Also note that the untimed `tryLock()` method does not honor the fairness setting. It will succeed if the lock is available even if other threads are waiting.

It is recommended practice to *always* immediately follow a call to `lock` with a `try` block, and to *always* immediately call `unlock` as the first statement in the finally block, as follows:

```
class X {
  private final ReentrantLock lock = new ReentrantLock();
  // ...

  public void m() {
    lock.lock();  // lock() as the last statement before the try block
    try {
```

```
      // ... method body
    } finally {
      lock.unlock(); // unlock() as the first statement in the finally block
    }
  }
}
```

In addition to implementing the `Lock` interface, this class defines a number of `public` and `protected` methods for inspecting the state of the lock. Some of these methods are only useful for instrumentation and monitoring.

Serialization of this class behaves in the same way as built-in locks: a deserialized lock is in the unlocked state, regardless of its state when serialized.

This lock supports a maximum of 2147483647 recursive locks by the same thread. Attempts to exceed this limit result in `Error` throws from locking methods.

Since:

1.5

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`ReentrantLock()`	Creates an instance of `ReentrantLock`.
`ReentrantLock(boolean fair)`	Creates an instance of `ReentrantLock` with the given fairness policy.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
`int`	`getHoldCount()`	Queries the number of holds on this lock by the current thread.

protected Thread	**getOwner**()	Returns the thread that currently owns this lock, or `null` if not owned.
protected Collection<Thread>	**getQueuedThreads**()	Returns a collection containing threads that may be waiting to acquire this lock.
final int	**getQueueLength**()	Returns an estimate of the number of threads waiting to acquire this lock.
protected Collection<Thread>	**getWaitingThreads**(Condition condition)	Returns a collection containing those threads that may be waiting on the given condition associated with this lock.
int	**getWaitQueueLength**(Condition condition)	Returns an estimate of the number of threads waiting on the given condition associated with this lock.
final boolean	**hasQueuedThread**(Thread thread)	Queries whether the given thread is waiting to acquire this lock.
final boolean	**hasQueuedThreads**()	Queries whether any threads are waiting to acquire this lock.
boolean	**hasWaiters**(Condition condition)	Queries whether any threads are waiting on the given condition associated with this lock.
final boolean	**isFair**()	Returns `true` if this lock has fairness set true.
boolean	**isHeldByCurrentThread**()	Queries if this lock is held by the current thread.
boolean	**isLocked**()	

		Queries if this lock is held by any thread.
void	**lock**()	Acquires the lock.
void	**lockInterruptibly**()	Acquires the lock unless the current thread is interrupted.
Condition	**newCondition**()	Returns a Condition instance for use with this Lock instance.
String	**toString**()	Returns a string identifying this lock, as well as its lock state.
boolean	**tryLock**()	Acquires the lock only if it is not held by another thread at the time of invocation.
boolean	**tryLock**(long timeout, TimeUnit unit)	Acquires the lock if it is not held by another thread within the given waiting time and the current thread has not been interrupted.
void	**unlock**()	Attempts to release this lock.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Details

ReentrantLock

```
public ReentrantLock()
```

Creates an instance of ReentrantLock. This is equivalent to using ReentrantLock(false).

ReentrantLock

```
public ReentrantLock(boolean fair)
```

Creates an instance of `ReentrantLock` with the given fairness policy.

Parameters:

`fair` - `true` if this lock should use a fair ordering policy

Method Details

lock

```
public void lock()
```

Acquires the lock.

Acquires the lock if it is not held by another thread and returns immediately, setting the lock hold count to one.

If the current thread already holds the lock then the hold count is incremented by one and the method returns immediately.

If the lock is held by another thread then the current thread becomes disabled for thread scheduling purposes and lies dormant until the lock has been acquired, at which time the lock hold count is set to one.

Specified by:

`lock` in interface `Lock`

lockInterruptibly

```
public void lockInterruptibly()
                      throws InterruptedException
```

Acquires the lock unless the current thread is interrupted.

Acquires the lock if it is not held by another thread and returns immediately, setting the lock hold count to one.

If the current thread already holds this lock then the hold count is incremented by one and the method returns immediately.

If the lock is held by another thread then the current thread becomes disabled for thread scheduling purposes and lies dormant until one of two things happens:

- The lock is acquired by the current thread; or
- Some other thread interrupts the current thread.

If the lock is acquired by the current thread then the lock hold count is set to one.

If the current thread:

- has its interrupted status set on entry to this method; or
- is interrupted while acquiring the lock,

then `InterruptedException` is thrown and the current thread's interrupted status is cleared.

In this implementation, as this method is an explicit interruption point, preference is given to responding to the interrupt over normal or reentrant acquisition of the lock.

Specified by:

`lockInterruptibly` in interface `Lock`

Throws:

`InterruptedException` - if the current thread is interrupted

tryLock

```
public boolean tryLock()
```

Acquires the lock only if it is not held by another thread at the time of invocation.

Acquires the lock if it is not held by another thread and returns immediately with the value `true`, setting the lock hold count to one. Even when this lock has been set to use a fair ordering policy, a call to `tryLock()` *will* immediately acquire the lock if it is available, whether or not other threads are currently waiting for the lock. This "barging" behavior can be useful in certain circumstances, even though it breaks fairness. If you want to honor the fairness setting for this lock, then use `tryLock(0, TimeUnit.SECONDS)` which is almost equivalent (it also detects interruption).

If the current thread already holds this lock then the hold count is incremented by one and the method returns `true`.

If the lock is held by another thread then this method will return immediately with the value `false`.

Specified by:

`tryLock` in interface `Lock`

Returns:

`true` if the lock was free and was acquired by the current thread, or the lock was already held by the current thread; and `false` otherwise

tryLock

```
public boolean tryLock(long timeout,
                       TimeUnit unit)
                throws InterruptedException
```

Acquires the lock if it is not held by another thread within the given waiting time and the current thread has not been interrupted.

Acquires the lock if it is not held by another thread and returns immediately with the value `true`, setting the lock hold count to one. If this lock has been set to use a fair ordering policy then an available lock *will not* be acquired if any other threads are waiting for the lock. This is in contrast to the `tryLock()` method. If you want a timed `tryLock` that does permit barging on a fair lock then combine the timed and un-timed forms together:

```
if (lock.tryLock() ||
    lock.tryLock(timeout, unit)) {
  ...
}
```

If the current thread already holds this lock then the hold count is incremented by one and the method returns `true`.

If the lock is held by another thread then the current thread becomes disabled for thread scheduling purposes and lies dormant until one of three things happens:

- The lock is acquired by the current thread; or
- Some other thread interrupts the current thread; or
- The specified waiting time elapses

If the lock is acquired then the value `true` is returned and the lock hold count is set to one.

If the current thread:

- has its interrupted status set on entry to this method; or
- is interrupted while acquiring the lock,

then `InterruptedException` is thrown and the current thread's interrupted status is cleared.

If the specified waiting time elapses then the value `false` is returned. If the time is less than or equal to zero, the method will not wait at all.

In this implementation, as this method is an explicit interruption point, preference is given to responding to the interrupt over normal or reentrant acquisition of the lock, and over reporting the elapse of the waiting time.

Specified by:

`tryLock` in interface `Lock`

Parameters:

`timeout` - the time to wait for the lock

`unit` - the time unit of the timeout argument

Returns:

`true` if the lock was free and was acquired by the current thread, or the lock was already held by the current thread; and `false` if the waiting time elapsed before the lock could be acquired

Throws:

`InterruptedException` - if the current thread is interrupted

`NullPointerException` - if the time unit is null

unlock

```
public void unlock()
```

Attempts to release this lock.

If the current thread is the holder of this lock then the hold count is decremented. If the hold count is now zero then the lock is released. If the current thread is not the holder of this lock then `IllegalMonitorStateException` is thrown.

Specified by:

`unlock` in interface `Lock`

Throws:

`IllegalMonitorStateException` - if the current thread does not hold this lock

newCondition

```
public Condition newCondition()
```

Returns a `Condition` instance for use with this `Lock` instance.

The returned `Condition` instance supports the same usages as do the `Object` monitor methods (`wait`, `notify`, and `notifyAll`) when used with the built-in monitor lock.

- If this lock is not held when any of the `Condition` waiting or signalling methods are called, then an `IllegalMonitorStateException` is thrown.
- When the condition waiting methods are called the lock is released and, before they return, the lock is reacquired and the lock hold count restored to what it was when the method was called.
- If a thread is interrupted while waiting then the wait will terminate, an `InterruptedException` will be thrown, and the thread's interrupted status will be cleared.
- Waiting threads are signalled in FIFO order.
- The ordering of lock reacquisition for threads returning from waiting methods is the same as for threads initially acquiring the lock, which is in the default case not specified, but for *fair* locks favors those threads that have been waiting the longest.

Specified by:

`newCondition` in interface `Lock`

Returns:

the Condition object

getHoldCount

```
public int getHoldCount()
```

Queries the number of holds on this lock by the current thread.

A thread has a hold on a lock for each lock action that is not matched by an unlock action.

The hold count information is typically only used for testing and debugging purposes. For example, if a certain section of code should not be entered with the lock already held then we can assert that fact:

```
class X {
  final ReentrantLock lock = new ReentrantLock();
  // ...
  public void m() {
    assert lock.getHoldCount() == 0;
    lock.lock();
    try {
      // ... method body
    } finally {
```

```
      lock.unlock();
    }
  }
}
```

Returns:

the number of holds on this lock by the current thread, or zero if this lock is not held by the current thread

isHeldByCurrentThread

`public boolean isHeldByCurrentThread()`

Queries if this lock is held by the current thread.

Analogous to the `Thread.holdsLock(Object)` method for built-in monitor locks, this method is typically used for debugging and testing. For example, a method that should only be called while a lock is held can assert that this is the case:

```
class X {
  final ReentrantLock lock = new ReentrantLock();
  // ...

  public void m() {
      assert lock.isHeldByCurrentThread();
      // ... method body
  }
}
```

It can also be used to ensure that a reentrant lock is used in a non-reentrant manner, for example:

```
class X {
  final ReentrantLock lock = new ReentrantLock();
  // ...

  public void m() {
      assert !lock.isHeldByCurrentThread();
      lock.lock();
      try {
          // ... method body
      } finally {
          lock.unlock();
```

```
      }
  }
}
```

Returns:

`true` if current thread holds this lock and `false` otherwise

isLocked

```
public boolean isLocked()
```

Queries if this lock is held by any thread. This method is designed for use in monitoring of the system state, not for synchronization control.

Returns:

`true` if any thread holds this lock and `false` otherwise

isFair

```
public final boolean isFair()
```

Returns `true` if this lock has fairness set true.

Returns:

`true` if this lock has fairness set true

getOwner

```
protected Thread getOwner()
```

Returns the thread that currently owns this lock, or `null` if not owned. When this method is called by a thread that is not the owner, the return value reflects a best-effort approximation of current lock status. For example, the owner may be momentarily `null` even if there are threads trying to acquire the lock but have not yet done so. This method is designed to facilitate construction of subclasses that provide more extensive lock monitoring facilities.

Returns:

the owner, or `null` if not owned

hasQueuedThreads

```
public final boolean hasQueuedThreads()
```

Queries whether any threads are waiting to acquire this lock. Note that because cancellations may occur at any time, a `true` return does not

guarantee that any other thread will ever acquire this lock. This method is designed primarily for use in monitoring of the system state.

Returns:

`true` if there may be other threads waiting to acquire the lock

hasQueuedThread

```
public final boolean hasQueuedThread(Thread thread)
```

Queries whether the given thread is waiting to acquire this lock. Note that because cancellations may occur at any time, a `true` return does not guarantee that this thread will ever acquire this lock. This method is designed primarily for use in monitoring of the system state.

Parameters:

`thread` - the thread

Returns:

`true` if the given thread is queued waiting for this lock

Throws:

`NullPointerException` - if the thread is null

getQueueLength

```
public final int getQueueLength()
```

Returns an estimate of the number of threads waiting to acquire this lock. The value is only an estimate because the number of threads may change dynamically while this method traverses internal data structures. This method is designed for use in monitoring system state, not for synchronization control.

Returns:

the estimated number of threads waiting for this lock

getQueuedThreads

```
protected Collection<Thread> getQueuedThreads()
```

Returns a collection containing threads that may be waiting to acquire this lock. Because the actual set of threads may change dynamically while constructing this result, the returned collection is only a best-effort estimate. The elements of the returned collection are in no particular order. This method is designed to facilitate construction of subclasses that provide more extensive monitoring facilities.

Returns:

the collection of threads

hasWaiters

```
public boolean hasWaiters(Condition condition)
```

Queries whether any threads are waiting on the given condition associated with this lock. Note that because timeouts and interrupts may occur at any time, a `true` return does not guarantee that a future `signal` will awaken any threads. This method is designed primarily for use in monitoring of the system state.

Parameters:

`condition` - the condition

Returns:

`true` if there are any waiting threads

Throws:

`IllegalMonitorStateException` - if this lock is not held

`IllegalArgumentException` - if the given condition is not associated with this lock

`NullPointerException` - if the condition is null

getWaitQueueLength

```
public int getWaitQueueLength(Condition condition)
```

Returns an estimate of the number of threads waiting on the given condition associated with this lock. Note that because timeouts and interrupts may occur at any time, the estimate serves only as an upper bound on the actual number of waiters. This method is designed for use in monitoring of the system state, not for synchronization control.

Parameters:

`condition` - the condition

Returns:

the estimated number of waiting threads

Throws:

`IllegalMonitorStateException` - if this lock is not held

`IllegalArgumentException` - if the given condition is not associated with this lock

`NullPointerException` - if the condition is null

getWaitingThreads

```
protected Collection<Thread> getWaitingThreads(Condition condition)
```

Returns a collection containing those threads that may be waiting on the given condition associated with this lock. Because the actual set of threads may change dynamically while constructing this result, the returned collection is only a best-effort estimate. The elements of the returned collection are in no particular order. This method is designed to facilitate construction of subclasses that provide more extensive condition monitoring facilities.

Parameters:

`condition` - the condition

Returns:

the collection of threads

Throws:

`IllegalMonitorStateException` - if this lock is not held

`IllegalArgumentException` - if the given condition is not associated with this lock

`NullPointerException` - if the condition is null

toString

```
public String toString()
```

Returns a string identifying this lock, as well as its lock state. The state, in brackets, includes either the String `"Unlocked"` or the String `"Locked by"` followed by the `name` of the owning thread.

Overrides:

`toString` in class `Object`

Returns:

a string identifying this lock, as well as its lock state

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.